

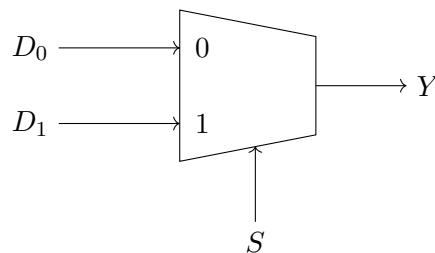
01 / AUSWÄHLEN STATT RECHNEN

Tag 9: Multiplexer

Ein Multiplexer (kurz MUX) wählt aus mehreren Dateneingängen einen aus und gibt dessen Wert am Ausgang weiter. Das Auswahlsignal bestimmt, **welcher Eingang** verbunden wird. Es ist nicht automatisch der Wert des Ausgangs.

Der kleinste Fall: 2:1-MUX

Zwei Dateneingänge heißen D_0 und D_1 . Ein Auswahl Eingang heißt S (Select). Der Ausgang heißt Y . Alle Signale sind hier ein Bit breit.



S	Y	Bedeutung
0	D_0	Nimm den Wert von Eingang 0.
1	D_1	Nimm den Wert von Eingang 1.

Ein Beispiel ganz langsam

Gegeben: $D_0 = 1$, $D_1 = 0$, $S = 0$.

1. Lies zuerst S : Es ist 0.
2. Wähle deshalb Eingang D_0 .
3. Lies dessen Wert: $D_0 = 1$. Also ist $Y = 1$.

Änderst du nur S auf 1, wird D_1 gewählt. Dann ist $Y = 0$. Der Eingang D_0 darf weiterhin 1 sein; er ist jetzt nicht ausgewählt.

Ein MUX fragt: **Welchen Datenwert soll ich weitergeben?** Die kleine 0 oder 1 am Symbol bezeichnet die Auswahl, nicht einen fest angeschlossenen Datenwert.

Wofür wird das gebraucht?

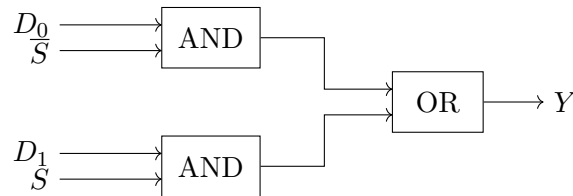
Ein Prozessor muss beispielsweise wählen, welches von zwei Rechenergebnissen weiterverarbeitet wird. Der MUX ist der auswählende Baustein. Er speichert dabei keinen Wert und braucht für seine kombinatorische Funktion keinen Takt.

02 / DER MUX AUS GATTERN

Warum diese Gleichung stimmt

$$Y = \bar{S}D_0 + SD_1$$

Die zwei UND-Terme lassen jeweils einen Datenweg durch. Das ODER führt die beiden Wege zum Ausgang zusammen.



$\bar{S}$ erzeugst du aus S mit einem NOT-Gatter. Es ist kein zweiter, unabhängiger Schalter.

Fall 1: S ist 0

$$Y = \bar{0}D_0 + 0D_1 = 1D_0 + 0 = D_0$$

Der obere Weg gibt D_0 weiter. Der untere Weg liefert immer 0.

Fall 2: S ist 1

$$Y = \bar{1}D_0 + 1D_1 = 0 + D_1 = D_1$$

Jetzt ist es umgekehrt. Es werden nicht beide Datenwerte miteinander verodert: Die Auswahl lässt nur einen der beiden Wege wirken.

Vollständige Wahrheitstabelle

S	D_0	D_1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Gegenprobe: Verändere nur den **nicht ausgewählten** Eingang. Bei stabiler Auswahl darf sich der stabile Ausgang dadurch nicht ändern.

03 / VIER DATENWEGE

4:1-MUX auf zwei Arten

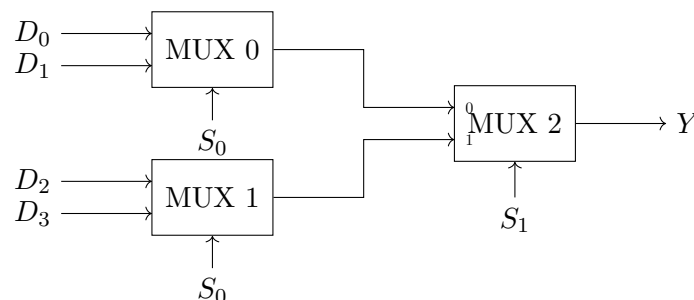
Vier Dateneingänge brauchen zwei Auswahlbits. Wir legen fest: S_1 ist das höherwertige Bit, S_0 das niederwertige. Hier gilt normale Binärreihenfolge, **keine Gray-Code-Reihenfolge**.

S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Aufbau direkt aus NOT, AND und OR

$$Y = \bar{S}_1 \bar{S}_0 D_0 + \bar{S}_1 S_0 D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3$$

Du brauchst zwei NOT-Gatter für die negierten Auswahlbits, vier AND-Gatter mit je drei Eingängen und ein OR-Gatter mit vier Eingängen. Jedes AND prüft eine Auswahlkombination und lässt dann seinen Datenwert durch. Bei $S_1 S_0 = 10$ wird nur der dritte Term aktiv: $Y = D_2$.

Aufbau aus drei 2:1-MUX

Zuerst wählt S_0 innerhalb beider Paare. Danach wählt S_1 zwischen den beiden Zwischenergebnissen. Für 10 wählen die ersten MUX D_0 und D_2 ; der letzte wählt D_2 .

Die zweimalige Beschriftung S_0 meint **denselben Eingang**. Im Simulator setzt du einen Eingabepin und verzweigst seine Leitung. Erzeuge keine zwei unabhängigen Eingaben gleichen Namens.

04 / EINE WAHRHEITSTABELLE VERDRAHTEN

Logik mit einem 8:1-MUX

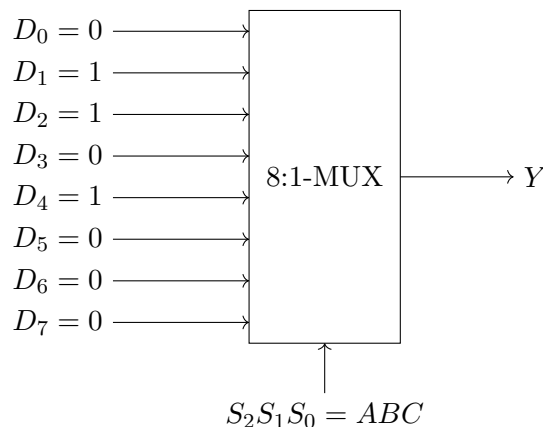
Ein 8:1-MUX hat acht Dateneingänge und drei Auswahlbits. Damit kannst du jede Boolesche Funktion mit drei Eingängen und einem Ausgang bauen: Die Eingangsvariablen wählen eine Tabellenzeile; am passenden Datenpin liegt deren Ausgangswert als Konstante.

Festgelegte Reihenfolge

Verbinde A mit S_2 , B mit S_1 und C mit S_0 . Dann ist der ausgewählte Index $4A + 2B + C$. Bei $ABC = 101$ wird D_5 ausgewählt.

Durchgerechnetes Beispiel: Y ist 1 bei genau einer Eingangs-1

A	B	C	Y	Datenpin mit diesem Wert
0	0	0	0	$D_0 = 0$
0	0	1	1	$D_1 = 1$
0	1	0	1	$D_2 = 1$
0	1	1	0	$D_3 = 0$
1	0	0	1	$D_4 = 1$
1	0	1	0	$D_5 = 0$
1	1	0	0	$D_6 = 0$
1	1	1	0	$D_7 = 0$



Die Datenpins werden mit Konstanten 0 oder 1 verbunden. Die veränderlichen Eingänge A , B und C gehen an die Auswahlpins. Ein Bus muss mit der vereinbarten Bitreihenfolge verdrahtet werden.

Optionaler Ausblick: ein kleinerer MUX

Bei einem 4:1-MUX kannst du A und B als Auswahl nehmen. Für jede AB -Kombination prüfst du Y bei $C = 0$ und $C = 1$: Das Paar 00 ergibt Datenwert 0, 11 ergibt 1, 01 ergibt C und 10 ergibt $\overline{C}$. Das ist Zusatzstoff, keine Pflichtaufgabe heute.

05 / EINE ADRESSE AKTIVIERT EINEN AUSGANG

Decoder: binär zu One-Hot

Ein 3:8-Decoder erhält drei Eingangsbits und hat acht Ausgänge. Für jede Adresse wird genau ein Ausgang 1; alle anderen sind 0. Wir verwenden aktive HIGH-Ausgänge, keinen Enable-Eingang und A_2 als höchstwertiges Bit.

$A_2A_1A_0$	Aktiver Ausgang	$Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$
000	Y_0	00000001
001	Y_1	00000010
010	Y_2	00000100
011	Y_3	00001000
100	Y_4	00010000
101	Y_5	00100000
110	Y_6	01000000
111	Y_7	10000000

Jeder Ausgang ist ein Minterm

Für Y_5 lautet die Adresse 101. Also soll der Ausgang genau dann 1 werden, wenn $A_2 = 1$, $A_1 = 0$ und $A_0 = 1$:

$$Y_5 = A_2 \bar{A}_1 A_0$$

Bei 101 ergibt das $1 \cdot 1 \cdot 1 = 1$. Bei jeder anderen Adresse ist mindestens ein Faktor 0. Genauso entstehen die sieben anderen Ausgänge. Zum Beispiel:

$$Y_0 = \bar{A}_2 \bar{A}_1 \bar{A}_0, \quad Y_3 = \bar{A}_2 A_1 A_0, \quad Y_7 = A_2 A_1 A_0$$

Aufbau und Verbindung zu SOP

Erzeuge die drei negierten Eingänge einmal mit NOT-Gattern. Baue dann acht AND-Gatter mit je drei Eingängen. Jedes bekommt die zu seinem Index passende Kombination aus normalen und negierten Eingängen.

Du kennst das schon von Tag 7: Jeder Ausgang steht für eine einzelne Tabellenzeile. Eine Funktion mit Einerindizes 1, 2 und 4 entsteht durch ein OR-Gatter an Y_1 , Y_2 und Y_4 .

Ein Decoder gibt **nicht die Binärzahl am Ausgang weiter**. Aus 101 wird eine Eins an Ausgang 5. Das heißt **One-Hot**: Genau ein Ausgang ist HIGH.

Unterschied zum MUX

Ein MUX wählt einen von mehreren vorhandenen **Datenwerten**. Ein Decoder aktiviert den zur Eingangsadresse gehörenden **Ausgang**. Er hat keine zusätzlichen Dateneingänge D_0 bis D_7 .

06 / WELCHER EINGANG HAT VORRANG?

8:3-Prioritätsencoder

Ein einfacher Encoder macht aus einem One-Hot-Eingang die Binärnummer des aktiven Eingangs. Was aber, wenn mehrere Eingänge gleichzeitig 1 sind? Ein **Prioritätsencoder** legt fest, wer gewinnt.

Unsere Regel: I_7 hat die höchste, I_0 die niedrigste Priorität. Die drei Ausgänge $Q_2Q_1Q_0$ enthalten den Index der höchsten aktiven Eingangsnummer.

$I_7 \dots I_0$	Gewinner	$Q_2Q_1Q_0$	NONE
00000010	I_1	001	0
00101000	I_5	101	0
10000001	I_7	111	0
00000001	I_0	000	0
00000000	keiner	000	1

Warum NONE nötig ist

Der Code 000 allein unterscheidet nicht zwischen „Eingang 0 gewinnt“ und „kein Eingang aktiv“. Deshalb hat die Schaltung einen zusätzlichen Ausgang:

$$\text{NONE} = \overline{I_7 + I_6 + I_5 + I_4 + I_3 + I_2 + I_1 + I_0}$$

Nur bei acht Nullen ist NONE gleich 1. Ein alternativ verwendetes VALID wäre das Gegenteil; wir bleiben hier durchgehend bei NONE.

Schritt 1: erst einen eindeutigen Gewinner bestimmen

Nenne G_k das Signal „Eingang k gewinnt“.

$$G_7 = I_7$$

$$G_6 = I_6 \bar{I}_7$$

$$G_5 = I_5 \overline{(I_7 + I_6)}$$

$$G_4 = I_4 \overline{(I_7 + I_6 + I_5)}$$

Für die niedrigeren Eingänge geht es genauso weiter: **Der eigene Eingang ist 1 UND alle höheren Eingänge sind 0**. Die Summe in der Klammer ist ein ODER, kein Rechnen mit Dezimalzahlen.

Beispiel 00101000: $I_5 = 1$ und $I_3 = 1$. G_5 wird 1, weil I_7 und I_6 beide 0 sind. G_3 bleibt 0, weil der höhere Eingang I_5 bereits aktiv ist.

Die Datenfolge wird in Tabellen immer als $I_7 \dots I_0$ geschrieben. Links steht die höchste Priorität. Das Ergebnis ist der **Index**, nicht die Anzahl der Einsen.

07 / VOM GEWINNER ZUM BINÄRCODE

Encoder aufbauen und prüfen

Schritt 2: Gewinnerleitungen codieren

Die Gewinnernummern 4, 5, 6 und 7 haben im Binärkode das höchste Bit 1. Deshalb ist Q_2 genau dann 1, wenn einer dieser Eingänge gewinnt. Dasselbe Prinzip gilt für die beiden anderen Bits:

$$Q_2 = G_4 + G_5 + G_6 + G_7$$

$$Q_1 = G_2 + G_3 + G_6 + G_7$$

$$Q_0 = G_1 + G_3 + G_5 + G_7$$

Für Gewinner 5 gilt: $5 = 101$. G_5 geht daher an das OR für Q_2 und an das OR für Q_0 , aber nicht an das OR für Q_1 .

Ein übersichtlicher Aufbau für die höheren Eingänge

Du kannst die ODER-Ausdrücke wiederverwenden. H_k bedeutet: „Mindestens ein Eingang mit höherem Index als k ist aktiv.“

$$H_6 = I_7, \quad H_5 = H_6 + I_6, \quad H_4 = H_5 + I_5, \quad \dots, \quad H_0 = H_1 + I_1$$

Danach ist $G_k = I_k \overline{H_k}$ für k von 0 bis 6; $G_7 = I_7$. Das ist ein gut lesbarer Aufbau, kein Anspruch auf die schnellste mögliche Schaltung.

Was du im Simulator prüfen solltest

- **4:1-MUX:** vier Auswahlzustände und alle 16 Datenkombinationen ergeben 64 Testfälle.
- **3:8-Decoder:** alle acht Eingangsadressen. Genau ein Ausgang muss aktiv sein.
- **8:3-Prioritätsencoder:** alle 256 Eingangsmuster, einschließlich keiner und mehrerer Einsen; NONE mitprüfen.
- **Logik mit 8:1-MUX:** alle acht Kombinationen von A, B und C gegen die gewünschte Wahrheitstabelle.

Im Praxisordner liegen CSV-Solltabellen für diese Prüfungen. Sie sind lesbare Vergleichsdaten, keine Zusage eines direkten Imports in jede Simulatorversion. Einzelne manuelle Beispiele sind ein Zwischentest; erst alle Zeilen belegen die vollständige Prüfung.

Praktische Stolperstellen

Setze jeden Eingang nur einmal und verzweige die Leitung. Eine Kreuzung ist nicht automatisch ein Verbindungspunkt. Stelle die Zahl der Gattereingänge passend ein und lasse keine vorgesehenen Eingänge offen. Bei MUX-Bausteinen prüfe Datenbreite, Auswahlbreite, Pinbeschriftung und ein eventuell vorhandenes Enable; alle hier verwendeten Datenleitungen sind 1 Bit breit.

Arbeitsumfang

Erst 2:1-MUX verstehen, dann 4:1-MUX, Decoder, Prioritätsencoder und die 8:1-Realisierung bearbeiten. Falls die Verdrahtung länger dauert, arbeite blockweise weiter. Die eigene QM-Implementierung gehört auf Samstag; heute gibt es keine neue Python-Pflichtaufgabe.

Buchbezug: DDCA 2.8, S. 81–86; Prioritätsencoder: Aufgabe 2.36, S. 100–101. Prüfablauf und Übungsdaten sind eigene Ergänzungen. Eigene Erklärungen und Beispiele.